



LabFlow

Pathology Lab Management System

PRODUCT DOCUMENTATION v1.0

Built with Laravel 12 + Vue 3 + MySQL
May 2026

LabFlow — Pathology Lab Management System

Product Documentation v1.0

Table of Contents

1. [Product Overview](#)
 2. [Technology Stack](#)
 3. [System Architecture](#)
 4. [Multi-Tenant Architecture](#)
 5. [User Roles & Permissions](#)
 6. [Application Flow — Master Flowchart](#)
 7. [Module Documentation](#)
 - [7.1 Authentication](#)
 - [7.2 Dashboard](#)
 - [7.3 Patients](#)
 - [7.4 Tests & Reference Ranges](#)
 - [7.5 Orders](#)
 - [7.6 Result Entry](#)
 - [7.7 PDF Report](#)
 - [7.8 Billing & Invoice](#)
 - [7.9 Lab Settings](#)
 - [7.10 Team Management](#)
 8. [Database Schema](#)
 9. [API Reference](#)
 10. [Frontend Routes](#)
 11. [Security Architecture](#)
 12. [Setup & Deployment](#)
-

1. Product Overview

LabFlow is a cloud-based, multi-tenant Pathology Lab Management System designed for independent diagnostic labs and small lab chains. Each lab operates in a fully isolated environment — one account per lab, with unlimited staff users under that account.

Core Capabilities

MODULE	WHAT IT DOES
Patient Registry	Register patients with demographics, auto-generate Patient UID
Test Catalog	Manage tests with categories, pricing, and per-parameter reference ranges
Order Management	Create test orders for patients, track status from pending → completed
Result Entry	Enter observed values per parameter; abnormal values auto-highlighted
PDF Report	Generate branded, patient-ready diagnostic reports with H/L flags
Billing	Create invoices with flat/percent discounts, record payments, download PDF
Settings	Manage lab profile and team members (admin only)

2. Technology Stack

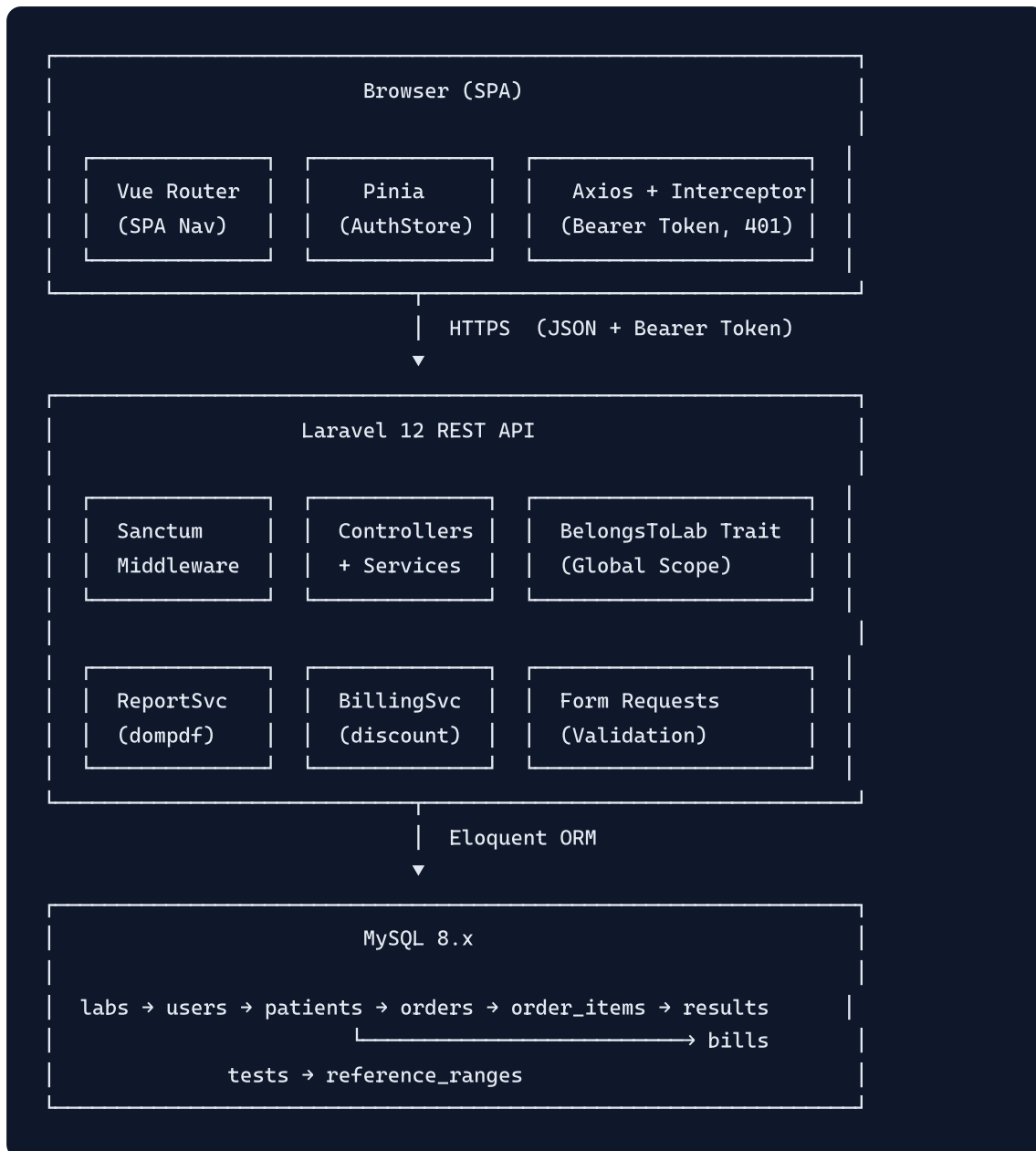
Backend

LAYER	TECHNOLOGY
Framework	Laravel 12 (PHP 8.2)
Authentication	Laravel Sanctum (Bearer tokens, 30-day expiry)
Database	MySQL 8.x
PDF Generation	barryvdh/laravel-dompdf
Testing	PHPUnit 11 + SQLite in-memory (71 tests)
Rate Limiting	Laravel built-in throttle middleware

Frontend

LAYER	TECHNOLOGY
Framework	Vue 3 (Composition API)
Build Tool	Vite
State Management	Pinia
HTTP Client	Axios
Styling	Tailwind CSS v3
Routing	Vue Router 4

3. System Architecture



4. Multi-Tenant Architecture

Every lab is a **tenant**. Data is isolated at the row level using a `lab_id` foreign key on every data table.

How It Works

```
User logs in
  |
  ▼
Sanctum issues token
  |
  ▼
Every API request carries: Authorization: Bearer {token}
  |
  ▼
auth:sanctum middleware resolves → $user (has lab_id)
  |
  ▼
BelongsToLab trait adds global Eloquent scope:
  WHERE table.lab_id = {user.lab_id}
  |
  ▼
All queries are automatically scoped – zero risk of cross-lab data leak
```

BelongsToLab Trait (applied to: Patient, Test, Order, Bill)

```
Model::query()
  └ Global Scope auto-adds: WHERE lab_id = current_user.lab_id
Model::create()
  └ creating event auto-sets: model.lab_id = current_user.lab_id
```

Tables with lab_id

TABLE	ISOLATION VIA
<code>users</code>	<code>lab_id</code> FK (direct member of lab)
<code>patients</code>	<code>lab_id</code> FK + BelongsToLab
<code>tests</code>	<code>lab_id</code> FK + BelongsToLab
<code>orders</code>	<code>lab_id</code> FK + BelongsToLab
<code>bills</code>	<code>lab_id</code> FK + BelongsToLab
<code>order_items</code>	through order → lab isolation
<code>results</code>	through order_item → lab isolation
<code>reference_ranges</code>	through test → lab isolation

5. User Roles & Permissions

ACTION	STAFF	ADMIN
View Dashboard	✓	✓
Register / View Patients	✓	✓
Create / Edit Tests	✓	✓
Create Orders	✓	✓
Enter Results	✓	✓
Download PDF Report	✓	✓
Create Bills & Record Payments	✓	✓
Download Invoice	✓	✓
Edit Lab Settings	✗	✓
Add / Remove Team Members	✗	✓

6. Application Flow — Master Flowchart

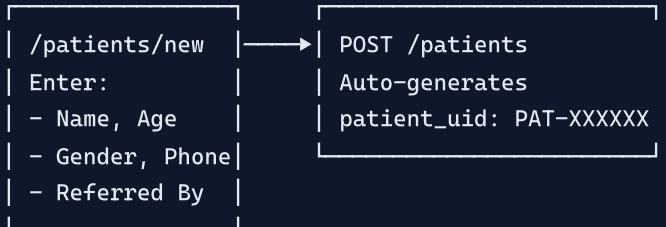
6.1 New Lab Onboarding



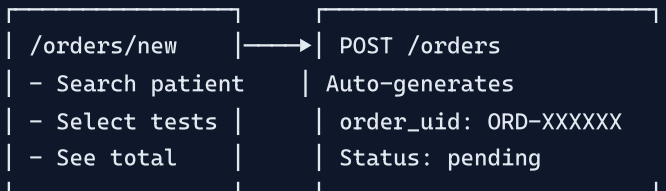
6.2 Daily Operation — Complete Patient Journey

COMPLETE LAB WORKFLOW

STEP 1: Register Patient



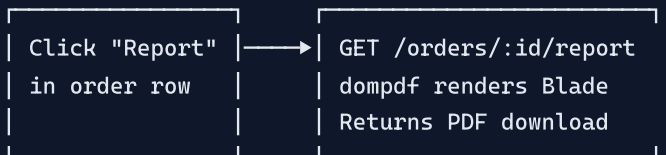
STEP 2: Create Order



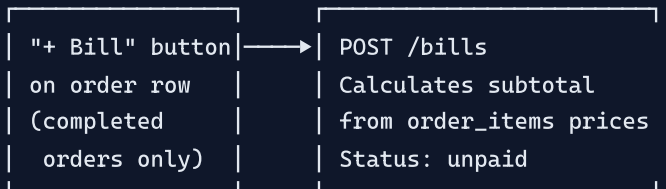
STEP 3: Enter Results



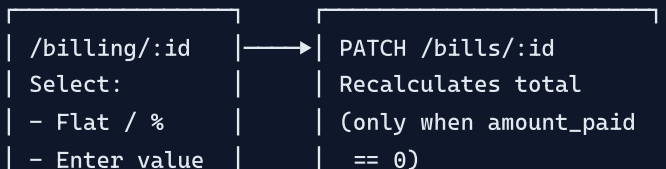
STEP 4: Download PDF Report



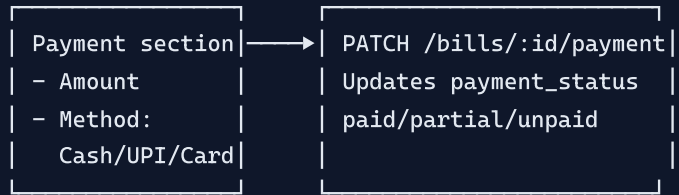
STEP 5: Create Bill



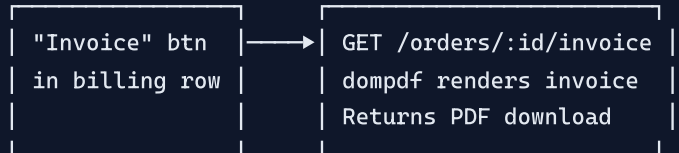
STEP 6: Apply Discount (optional)



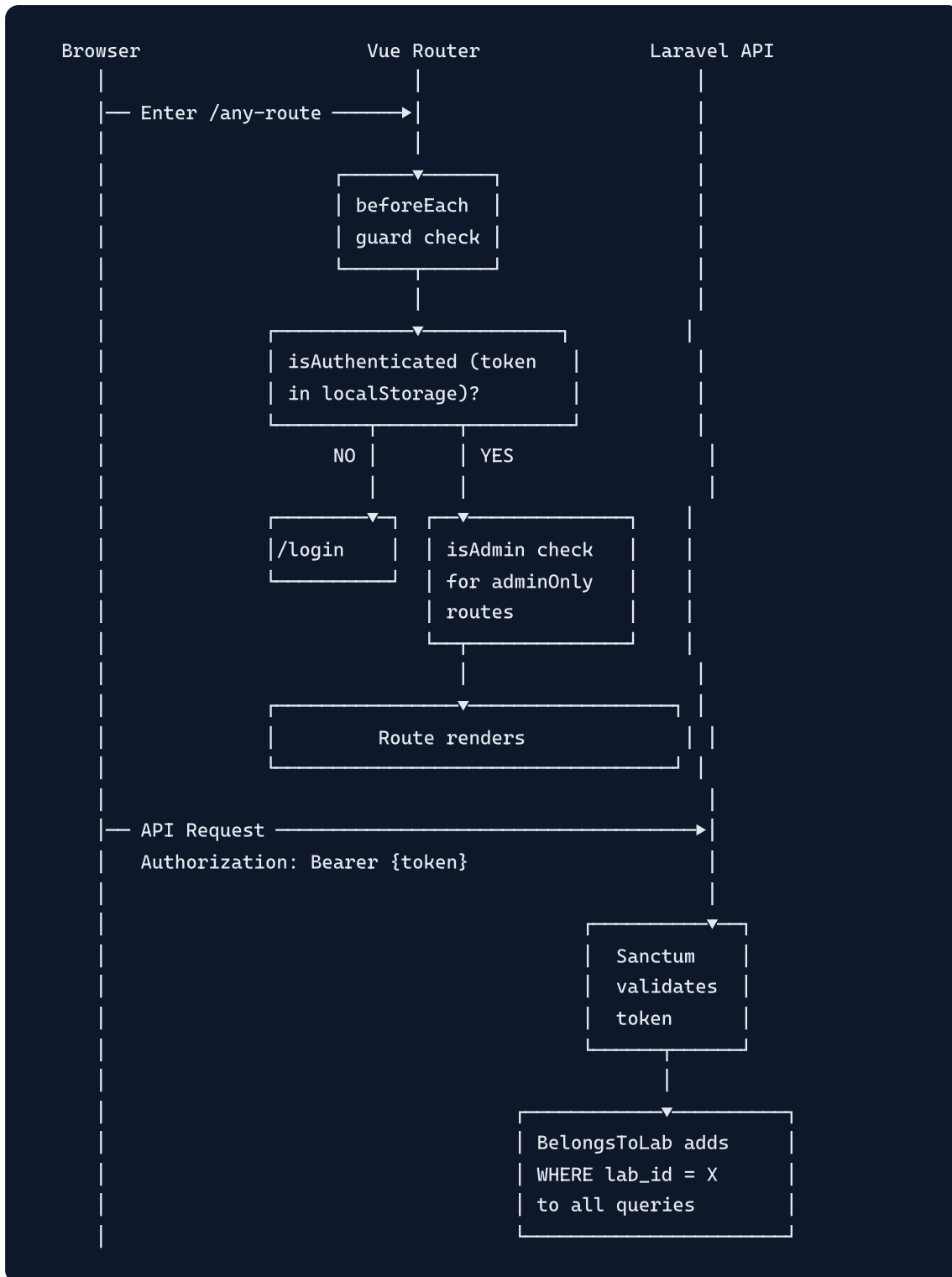
STEP 7: Record Payment



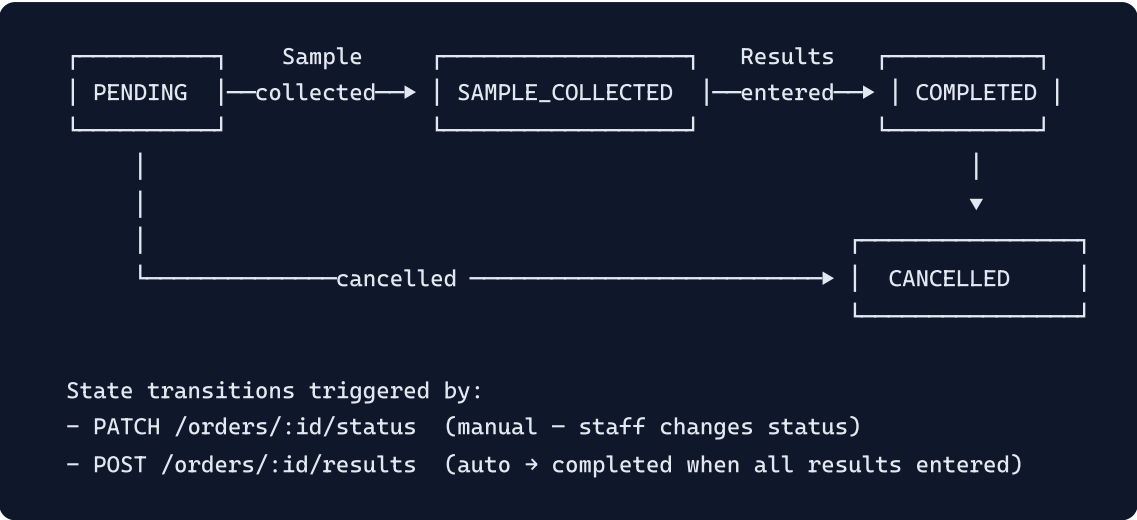
STEP 8: Download Invoice



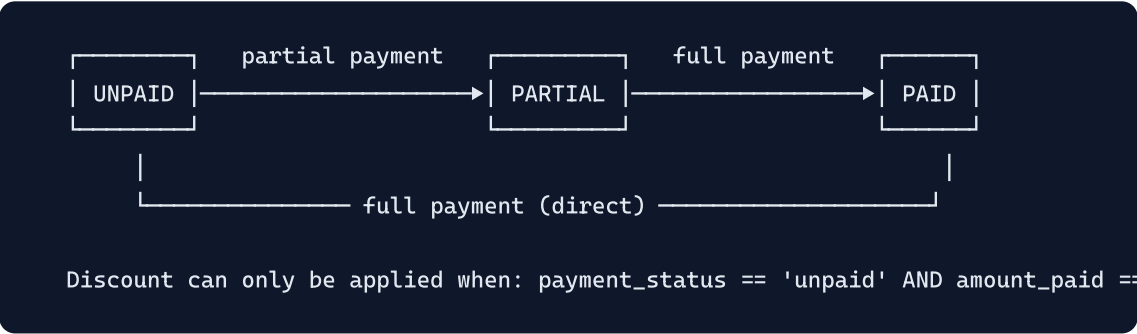
6.3 Authentication Flow



6.4 Order Status State Machine



6.5 Bill Payment State Machine



7. Module Documentation

7.1 Authentication

Pages: </login> , </register>

ACTION	ENDPOINT	RATE LIMIT
Register new lab	POST /api/v1/auth/register	10 req/min
Login	POST /api/v1/auth/login	10 req/min
Logout	POST /api/v1/auth/logout	—
Get current user	GET /api/v1/auth/me	—

Token Storage: [localStorage](#) keys: [auth_token](#) , [auth_user](#) , [auth_lab](#)

Token Expiry: 30 days (automatic)

Register Payload:

```
{
  "lab_name": "City Diagnostics",
  "lab_email": "info@citydiag.com",
  "lab_phone": "9876543210",
  "name": "Dr. Ramesh Kumar",
  "email": "dr.ramesh@citydiag.com",
  "password": "password123",
  "password_confirmation": "password123"
}
```

7.2 Dashboard

Page: /

Displays a real-time snapshot of lab activity.

CARD	DATA SOURCE
Patients Today	Count of patients created today
Orders Today	Count of orders created today
Pending Orders	Count of orders with status = pending/sample_collected
Unpaid Bills	Count of bills with payment_status = unpaid/partial
Revenue Today	Sum of amount_paid from bills paid today
Revenue This Month	Sum of amount_paid from bills paid this month
Total Patients	All-time patient count
Completed Orders	All-time completed order count

Recent Orders Table (last 8): Patient name, UID, status badge, payment status, links to Results/Bill/PDF.

7.3 Patients

Pages: [/patients](#) , [/patients/new](#) , [/patients/:id](#) , [/patients/:id/edit](#)

Patient UID Format: [PAT-XXXXXX](#) (auto-generated, unique per lab)

Fields:

FIELD	REQUIRED	NOTES
name	Yes	Full name
age	Yes	Numeric
age_unit	No	years/months/days
gender	Yes	male / female / other
phone	No	Contact number
email	No	Email address
address	No	Postal address
referred_by	No	Doctor name

PatientDetail page shows complete order history with status, bill status, and actions.

7.4 Tests & Reference Ranges

Pages: [/tests](#) , [/tests/new](#) , [/tests/:id/edit](#)

Test Fields:

FIELD	REQUIRED	NOTES
test_code	Yes	e.g. CBC001, LFT001
test_name	Yes	e.g. Complete Blood Count
category	No	Haematology / Biochemistry etc.
sample_type	Yes	blood / urine / stool / swab / other
price	Yes	Default price for billing
turnaround_hours	No	Default: 24 hrs
is_active	—	Default: true

Reference Ranges (per test, per parameter):

FIELD	NOTES
parameter_name	e.g. Haemoglobin, WBC
unit	e.g. g/dL, 10 ³ /μL
min_value / max_value	Numeric range (optional)
text_range	Free text range (e.g. "Negative")
gender_filter	all / male / female
age_min / age_max	Age-based range filter

7.5 Orders

Pages: </orders> , </orders/new>

NewOrder Flow:

1. Search patient by name/phone (typeahead)
2. Select one or more tests from catalog
3. Running price total updates live
4. Submit → creates Order + OrderItems

OrderList Actions by Status:

ORDER STATUS	AVAILABLE ACTIONS
pending	View Results, (no bill yet)
sample_collected	View Results
completed	View Results, Download Report, + Bill (if no bill exists)
completed + bill	View Results, Download Report, View Bill
cancelled	—

7.6 Result Entry

Page: `/orders/:id`

- Displays a grid: one row per test parameter across all ordered tests
- Each row shows: Parameter Name | Unit | Reference Range | Value Input | Abnormal Flag
- Abnormal detection: compares entered value against reference ranges filtered by patient's age and gender
- Values outside range are highlighted in red with H (High) / L (Low) flag
- On submit: calls `POST /orders/:id/results` with all parameter values
- Order status auto-advances to `completed`

7.7 PDF Report

Trigger: `GET /api/v1/orders/:id/report`

Report Layout:

[Lab Logo/Name]		[Lab Address & Contact]		
Patient: John Doe		UID: PAT-000123		
Age: 45Y / Male		Collected: 07-May-26		
Referred by: Dr. X		Report: 07-May-26		
Test / Param	Value	Unit	Range	F
CBC				
Haemoglobin	**9.2**	g/dL	13-17	L
WBC	7.5	10 ³ /μL	4-11	
[Lab Signature / Seal Line]				
Page 1/1				

← red bold

Generated via [barryvdh/laravel-dompdf](#) , A4, streamed as download.

7.8 Billing & Invoice

Pages: [/billing](#) , [/billing/:id](#)

Bill Creation: Triggered by "+ Bill" button on completed orders. Auto-calculates subtotal from sum of [price_at_order](#) across all order items.

Discount Types:

- [flat](#) — fixed rupee deduction (e.g. ₹50 off)
- [percent](#) — percentage deduction (e.g. 10% off)

Payment Methods: cash / upi / card / other

Payment Status Flow: unpaid → partial → paid

Invoice PDF: [GET /api/v1/orders/:id/invoice](#) — branded invoice with patient, tests, discount breakdown, payment summary.

7.9 Lab Settings (Admin Only)

Page: [/settings/lab](#)

Editable lab profile fields: Name, Email, Phone, Address, Registration/License No.

Changes are reflected immediately in the sidebar lab name and the PDF report header.

7.10 Team Management (Admin Only)

Page: </settings/users>

Add Team Member — Fields: Full Name, Email, Password, Confirm Password, Role (Staff / Admin)

Team Table — Lists all users in the lab with Name, Email, Role badge, Join Date, Remove button.

Rules:

- Admin cannot remove their own account
 - New users are immediately active (no email verification in v1)
 - All users created under the same [lab_id](#) — inherit full lab data access
-

8. Database Schema

Entity Relationship Diagram

```
labs
├── id, name, email, phone, address, registration_no, is_active
├── users
│   ├── id, lab_id, name, email, password, role (admin|staff)
├── patients
│   ├── id, lab_id, patient_uid, name, age, age_unit, gender,
│   │   phone, email, address, referred_by
├── tests
│   ├── id, lab_id, test_code, test_name, category, sample_type,
│   │   price, turnaround_hours, is_active
│   └── reference_ranges
│       ├── id, test_id, parameter_name, unit, min_value,
│       │   max_value, text_range, gender_filter, age_min, age_max
└── orders (via patients.id)
    ├── id, lab_id, patient_id, order_uid, ordered_at, status
    ├── order_items
    │   ├── id, order_id, test_id, price_at_order, status
    │   └── results
    │       ├── id, order_item_id, parameter_name,
    │       │   observed_value, unit, is_abnormal, remarks,
    │       │   entered_by
    └── bills
        ├── id, lab_id, order_id, bill_uid, subtotal,
        │   discount_type, discount_value, total_amount,
        │   payment_status, amount_paid, payment_method
```

Order Status Enum

pending | sample_collected | result_entered | completed | cancelled

Bill Payment Status Enum

unpaid | partial | paid

Sample Type Enum

blood | urine | stool | swab | other

9. API Reference

Base URL: `http://your-domain.com/api/v1`

All protected endpoints require: `Authorization: Bearer {token}`

Authentication

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	<code>/auth/register</code>	Public	Register new lab + admin
POST	<code>/auth/login</code>	Public	Login, returns token
POST	<code>/auth/logout</code>	Bearer	Revoke current token
GET	<code>/auth/me</code>	Bearer	Get current user + lab

Lab & Users

METHOD	ENDPOINT	ROLE	DESCRIPTION
GET	<code>/lab</code>	Any	Get current lab info
PATCH	<code>/lab</code>	Admin	Update lab settings
GET	<code>/users</code>	Any	List lab team members
POST	<code>/users</code>	Admin	Add team member
DELETE	<code>/users/:id</code>	Admin	Remove team member

Patients

METHOD	ENDPOINT	DESCRIPTION
GET	<code>/patients</code>	Paginated list (search by name/phone)
POST	<code>/patients</code>	Create patient
GET	<code>/patients/:id</code>	Patient + order history
PUT	<code>/patients/:id</code>	Update patient
DELETE	<code>/patients/:id</code>	Soft delete

Tests

METHOD	ENDPOINT	DESCRIPTION
GET	/tests	List tests (filter by category)
POST	/tests	Create test
GET	/tests/:id	Test + reference ranges
PUT	/tests/:id	Update test
DELETE	/tests/:id	Soft delete
POST	/tests/:id/ranges	Add reference range
PUT	/tests/:id/ranges/:rid	Update range
DELETE	/tests/:id/ranges/:rid	Delete range

Orders

METHOD	ENDPOINT	DESCRIPTION
GET	/orders	List (filter by status/date)
POST	/orders	Create order with test items
GET	/orders/:id	Order + items + results
PATCH	/orders/:id/status	Update status
DELETE	/orders/:id	Cancel/delete order

Results & Reports

METHOD	ENDPOINT	DESCRIPTION
POST	/orders/:id/results	Bulk enter all results
PUT	/results/:id	Update single result
GET	/orders/:id/report	Download PDF report
GET	/orders/:id/invoice	Download PDF invoice

Billing

METHOD	ENDPOINT	DESCRIPTION
GET	/bills	List bills (filter by payment status)
POST	/bills	Create bill for order
GET	/bills/:id	Bill + order + patient detail
PATCH	/bills/:id	Apply discount
PATCH	/bills/:id/payment	Record payment

Dashboard

METHOD	ENDPOINT	DESCRIPTION
GET	/dashboard/stats	KPI stats + recent orders

10. Frontend Routes

ROUTE	PAGE	AUTH	ROLE
/login	Login.vue	Public	—
/register	Register.vue	Public	—
/	Dashboard.vue	Auth	Any
/patients	PatientList.vue	Auth	Any
/patients/new	PatientForm.vue	Auth	Any
/patients/:id	PatientDetail.vue	Auth	Any
/patients/:id/edit	PatientForm.vue	Auth	Any
/tests	TestList.vue	Auth	Any
/tests/new	TestForm.vue	Auth	Any
/tests/:id/edit	TestForm.vue	Auth	Any
/orders	OrderList.vue	Auth	Any
/orders/new	NewOrder.vue	Auth	Any
/orders/:id	ResultEntry.vue	Auth	Any
/billing	BillList.vue	Auth	Any
/billing/:id	BillDetail.vue	Auth	Any
/settings/lab	LabSettings.vue	Auth	Admin only
/settings/users	UserManagement.vue	Auth	Admin only

11. Security Architecture

Authentication

- **Sanctum Bearer Tokens** — stateless, no cookies, safe for API-only use
- **30-day expiry** — tokens auto-expire, user must re-login

- **Token revocation** — logout calls `DELETE /tokens/current` server-side
- **Rate limiting** — `/auth/register` and `/auth/login` limited to 10 requests/minute per IP

Data Isolation

- **Row-level multi-tenancy** — `lab_id` on all data tables
- **Eloquent global scope** — `BelongsToLab` trait, auto-applied, cannot be forgotten
- **No cross-lab queries possible** — even if an attacker has a valid token from Lab A, they cannot access Lab B data
- **admin-only routes** — `UserManagement` , `LabSettings` guarded by both frontend (router guard) and backend (middleware check in controller)

Frontend Security

- **401 interceptor** — any 401 from API clears localStorage and redirects to `/login`
- **Route guards** — unauthenticated users redirected to `/login` ; non-admins redirected from admin routes
- **No sensitive data in URL** — tokens stored in localStorage, never in query strings

Password Security

- Passwords hashed with **bcrypt** (Laravel default)
- Minimum 8 characters enforced via validation
- Password confirmation required on register and when adding team members

12. Setup & Deployment

Prerequisites

- PHP 8.2+ with extensions: pdo_mysql, mbstring, xml, fileinfo
- MySQL 8.x
- Node.js 18+ and npm
- Composer 2.x

Backend Setup

```
# 1. Clone and install
cd pathology-lab
composer install

# 2. Configure environment
cp .env.example .env
php artisan key:generate

# Edit .env:
# DB_DATABASE=pathology_lab
# DB_USERNAME=root
# DB_PASSWORD=your_password

# 3. Run migrations
php artisan migrate

# 4. (Optional) Seed sample tests
php artisan db:seed --class=TestSeeder

# 5. Start server
php artisan serve # runs on http://localhost:8000
```

Frontend Setup

```
cd pathology-lab/frontend

# 1. Install dependencies
npm install

# 2. Configure API URL
# Edit .env:
# VITE_API_BASE_URL=http://localhost:8000/api

# 3. Start dev server
npm run dev # runs on http://localhost:5173

# 4. Production build
npm run build
```

First-Time Use

1. Open <http://localhost:5173/register>
2. Fill in your lab name and create the admin account
3. Login with the admin credentials
4. Go to **Tests** → Add your test catalog with reference ranges

5. Start registering patients and creating orders

Environment Variables

Backend (`.env`)

```
APP_NAME=LabFlow
APP_URL=http://localhost:8000
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=pathology_lab
DB_USERNAME=root
DB_PASSWORD=
SANCTUM_STATEFUL_DOMAINS=localhost:5173
SESSION_DOMAIN=localhost
```

Frontend (`frontend/.env`)

```
VITE_API_BASE_URL=http://localhost:8000/api
```

LabFlow v1.0 — Built with Laravel 12 + Vue 3